



Advanced Terraform on Azure

Lab 7 - Change Governance

Part 1. Classifying Change	2
Overview	2
Starting State	2
Objective	2
Defining Change Intent	3
Promotion Attempt 1: PROMOTION INTENT	5
Promotion Attempt 2: IMPLICIT PROMOTION ATTEMPT	6
Promotion Attempt 3: NO-OP PROMOTION INTENT	7
Non-Promotion Update Attempt 1 : ENV ROOT CHANGE	9
Comparison with Lab 6b	10
Reflection	10
Part 2. Change Authorisation	11
Governance Model	11
Enforce Authorisation in the Pipeline	12
Test 1 - Attempt an Unauthorised Promotion	13
Test 2 - Perform an Authorised Promotion	14
Run your own tests (Optional)	16
Shift governance out of main.....	17
Test governance before hitting main.....	18
Key Takeaway.....	19
Part 3 - Governance-of-Governance	20
Phase 1 – Introduce the Governance Pipeline	20
Phase 2 – Enforce Governance-of-Governance	21
Phase 3 – Break-Glass Recovery	23
Phase 4 – Normal Governance Operation	23
Key Learning Points	24

Part 1. Classifying Change

Overview

In the previous lab you performed a real production promotion using a robust, environment-scoped pipeline architecture.

Everything worked. The promotion was:

- deliberate
- approved
- controlled
- auditable in execution

And yet, the most important question remained unanswered:

Where was the decision to change an environment explicitly declared?

In this lab, you will **not add more automation**.

You will **not improve Terraform logic**.

You will **not increase safety controls**.

Instead, you will redesign the promotion flow so that automation is no longer allowed to infer intent. Configuration change intent must be:

- explicit
- deliberate
- durable
- unambiguous

And automation must **refuse to act** unless that intent exists.

Starting State

You must have completed **Lab 6b** successfully.

At this point:

- Environment-scoped pipelines are in place
- Production promotion works
- Approval gates exist
- Terraform applies cleanly
- The audit gap has been observed and documented

You are not fixing a broken system. You are fixing an **incomplete decision model**.

If you catch yourself thinking “We could just infer this from X” Pause. That is exactly the behaviour this lab removes.

Objective

By the end of this lab, you should be able to say - with evidence:

“Environment-level-only changes are classified as such. Module-level changes are classified as promotions and happen because they have been explicitly declared, automation refusing to run until that declaration exists.”

Defining Change Intent

The blast-radius difference between changes to existing modules is significantly smaller than changing the actual modules being used. The intended level of change should therefore be defined and ascertained prior to being performed. High risk module changes justify explicit intent artifacts that state the intended module changes and rationale.

The four change types introduced in this phase are ...

Change Type	What the Human Did	What the Pipeline Concludes
ENV_ROOT_CHANGE	Changed environment config only (no module pin changes)	Local environment modification
PROMOTION_INTENT	Changed module pinned version and declared matching intent	Explicit, approved promotion
IMPLICIT_PROMOTION_ATTEMPT	Changed a module pinned version without declaring promotion intent	Tried to promote shared code by stealth
NO-OP_PROMOTION_INTENT	Declared promotion intent without changing any module version pinning	Recorded intent with no effective action

You will now introduce new promotion-intent declaration files and update the existing change-env pipeline yaml files.

The promotion declaration file:

- exists in version control
- is human-authored
- is environment-specific
- carries no executable logic

The updated pipeline:

- Enforces checks against the promotion-intent declaration file.

Move the intent artifacts into the env-roots repo

1. Copy from lab artifacts into the env-roots repo:

- From: artifacts\tfvars\promotion\prod-promotion.tfvars
To: env-roots\envs\prod\
- From: artifacts\tfvars\promotion\test-promotion.tfvars
To: env-roots\envs\test\
- From: artifacts\tfvars\dev-promotion.tfvars
To: env-roots\envs\dev\

2. At **env-roots** run ...

```
cd c:\ado-repo\env-roots
git checkout main
git pull
git branch -D intent-files-upload
git checkout -b intent-files-upload
git add envs\prod\prod-promotion.tfvars
git add envs\test\test-promotion.tfvars
git add envs\dev\dev-promotion.tfvars
git commit -m "Introduce intent declaration files"
git push -u origin intent-files-upload
```

3. Add the PR ...

This step triggers the optional {env}-plan pipeline runs prior to merging to main.

4. **Complete/Merge** the PR once the plan pipelines complete. This merges the files to main and triggers the change-{env} pipelines to run. Allow these **no-ops** to run, granting permissions and approvals as required..

5. The change-{env} pipeline files have three feature levels. Level 1, the current level, is Terraform only. Level 2 is Terraform and Intent and this will now enabled.

Locally, update the three **change-{env}.yml** files in **env-roots\azure-pipelines**, changing the **FEATURE_PHASE** value from "1" to "2", thus enabling Intent logic.

6. At env-roots run ...

```
cd c:\ado-repo\env-roots
git checkout main
git pull
git branch -D intent-gate
git checkout -b intent-gate
git add .azure-pipelines\change-*.yml
git commit -m "Introduce intent gate"
git push -u origin intent-gate
```

7. Add the PR and Complete/merge to main

Rollout v1.0.2 of security module

8. In the **terraform-azure-module-security** module repository:

- Open the existing locals.tf
- Update the manager value from 'Peter Smith (SecMod v1.0.1)' to '**Oliver Walsh (SecMod v1.0.2)**'

This is a deliberately boring change. The change itself is not the point.

9. Commit and tag

At **terraform-azure-module-security** run ...

```
cd c:\module-repos\terraform-azure-module-security
git add locals.tf
git commit -m "Update manager value"
git push origin main
git tag v1.0.2
git push origin v1.0.2
```

Confirm that the new tag exists remotely.

Promotion Attempt 1: PROMOTION INTENT

You will now perform a promotion, as you did in the previous lab - but this time, with intent, declared **before** execution. You will promote a new **security** module version to **PROD**.

Expected result: SUCCESS - **PROMOTION_INTENT**

Make a module configuration change and declare intent

10. Edit **envs/prod/prod-promotion.tfvars**

Promotion Reason: **Security Update**

Approved Refs:

Network v1.0.2 (no change)

Security **v1.0.2** (update)

11. Edit **envs/prod/main.tf**

- Set the pinned security module version to **v1.0.2**

12. Push the changes ...

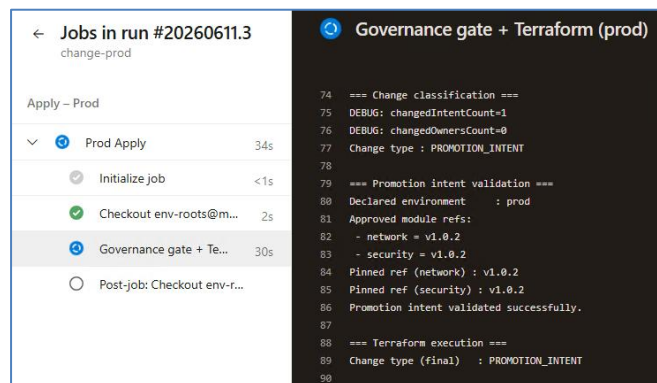
```
cd c:\ado-repo\env-roots
git checkout main
git pull
git branch -D promote-prod-1
git checkout -b promote-prod-1
```

```
git add envs\prod\prod-promotion.tfvars
git add envs\prod\main.tf
git commit -m "Declare intent and promote SECv1.0.2 to PROD"
git push -u origin promote-prod-1
```

13. Raise a PR.

Observe the Pipeline Behaviour

14. Complete\Merge the PR after the auto-triggered **pr-plan-prod** planning completes.
15. Approve the run and carefully observe as the **change-prod** pipeline now reads declared intent and validates it
16. If the declaration is missing or incorrect the pipeline stops **by design** otherwise it passes through the gate ...



II Pause – Question 3

17. Ask yourself:

- Where is the promotion decision now recorded?
- Is it inferred?
- Or explicitly stated?

Promotion Attempt 2: IMPLICIT PROMOTION ATTEMPT

Attempt a promotion without explicit intent declaration. Prove that changing configuration alone is not enough. Promotion must be also declared explicitly in the intent file.

Expected result: FAILURE **IMPLICIT_PROMOTION_ATTEMPT**

Make a module configuration change without declaring intent

18. Edit **envs/prod/main.tf**

- Change the pinned security module version from v1.0.2 to **v1.0.1**

19. Do not edit **envs/prod/prod-promotion.tfvars**

20. Push the changes ...

```

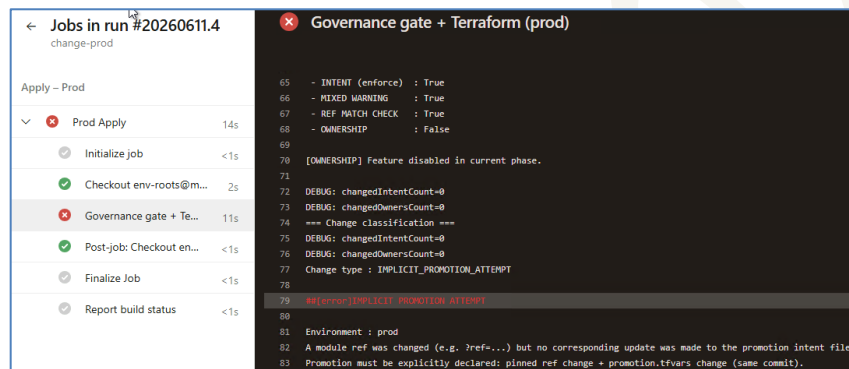
cd c:\ado-repo\env-roots
git checkout main
git pull
git branch -D promote-prod-2
git checkout -b promote-prod-2
git add envs\prod\main.tf
git commit -m "Promote SECv1.0.1 to PROD"
git push -u origin promote-prod-2

```

21. Merge the PR, observe planning, then complete/merge and Approve.

Observe (Expected Result)

After completing or skipping the auto-triggered **pr-plan-prod** pipeline, completing (merging) the PR triggers the **change-prod** pipeline which performs intent validation. If intent (prod-promotion.tfvars) is missing or does not match what is pinned in main.tf, the promote pipeline stops by design ...



22. The code change merged to main before the promote failure, so roll back the failure :

```

cd c:\ado-repo\env-roots
git checkout main
git pull
git revert -m 1 $(git rev-list --merges -n 1 HEAD) --no-edit
git push

```

23. This triggers a no-op change-prod pipeline run which can be cancelled or allowed to complete.

Promotion Attempt 3: NO-OP PROMOTION INTENT

Prove that declared promotion intent must also be accompanied by the appropriate configuration change.

Expected result: FAILURE **NO-OP_PROMOTION_INTENT**

Make a module configuration change without declaring intent

24. Edit `envs/prod/prod-promotion.tfvars`

Promotion Reason: **Security Update**

Approved Refs:

Network v1.0.2 (no change)

Security **v1.0.1** (update)

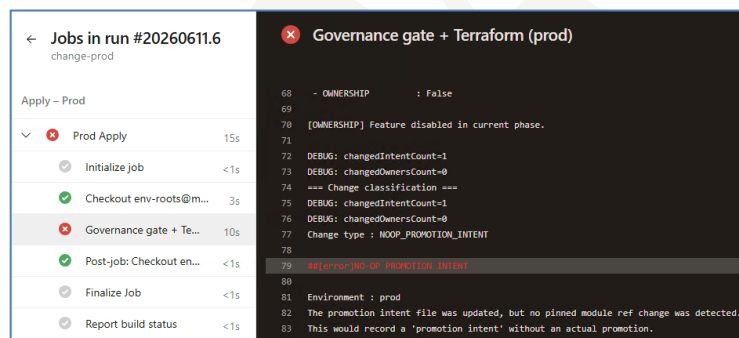
25. Create a new local branch and push changes...

```
cd c:\ado-repo\env-roots
git checkout main
git pull
git branch -D promote-prod-3
git checkout -b promote-prod-3
git add envs\prod\prod-promotion.tfvars
git commit -m "Promote SECv1.0.1 to PROD"
git push -u origin promote-prod-3
```

26. Merge the PR, observe planning, then complete/merge and approve.

Observe (Expected Result)

The **prod-plan** pipeline triggers. Completing (merging) triggers the **prod-promote** pipeline which performs intent validation. If intent (`prod-promotion.tfvars`) is missing or does not match what is pinned in `main.tf`, the change pipeline stops by design ...



27. Again, the code change merged to main before the change failure, so roll back the code change :

```
cd c:\ado-repo\env-roots
git checkout main
git pull
git revert -m 1 $(git rev-list --merges -n 1 HEAD) --no-edit
git push
```

28. This triggers a no-op change-prod pipeline run which can be cancelled or allowed to complete.

Non-Promotion Update Attempt 1 : ENV ROOT CHANGE

Prove that a configuration change that does not include a module promotion is classified as an environment-level change.

Expected result: SUCCESS **ENV_ROOT_CHANGE**

Make an environment level change

29. Edit **envs/prod/terraform.tfvars**

Change the **data** subnet CIDR to "**10.2.99.0/24**"

30. Push the changes ...

```
cd c:\ado-repo\env-roots
git checkout main
git pull
git branch -D change-prod-env-1
git checkout -b change-prod-env-1
git add envs\prod\terraform.tfvars
git commit -m "Non-module update"
git push -u origin change-prod-env-1
```

31. Merge the PR, observe planning, then complete/merge and Approve.

Observe (Expected Result)

The **prod-plan** pipeline triggers. Completing (merging) triggers the **prod-change** pipeline which performs intent validation. If intent (prod-promotion.tfvars) is missing or does not match what is pinned in main.tf, the promote pipeline stops by design. In this case there are no module changes being proposed...

The screenshot displays the 'Jobs in run #20260611.8' for the 'change-prod' pipeline. The job list on the left shows 'Prod Apply' (40s), 'Initialize job' (<1s), 'Checkout env-roots@m...' (1s), 'Governance gate + Terraform (prod)' (37s), 'Post-job: Checkout en...' (<1s), 'Finalize Job' (<1s), and 'Report build status' (<1s). The 'Governance gate + Terraform (prod)' job is selected, showing its console output on the right. The output includes debug messages, Terraform execution details, and a warning about Terraform version 1.10.5 being out of date compared to 1.15.6.

Job Name	Status	Duration
Prod Apply	Completed	40s
Initialize job	Completed	<1s
Checkout env-roots@m...	Completed	1s
Governance gate + Terraform (prod)	Completed	37s
Post-job: Checkout en...	Completed	<1s
Finalize Job	Completed	<1s
Report build status	Completed	<1s

```
76 DEBUG: changedOwnersCount=0
77 Change type : ENV_ROOT_CHANGE
78
79 === Terraform execution ===
80 Change type (final) : ENV_ROOT_CHANGE
81
82 Terraform v1.10.5
83 on windows_amd64
84 + provider registry.terraform.io/hashicorp/azurerm v4.76.0
85
86 Your version of Terraform is out of date! The latest version
87 is 1.15.6. You can update by downloading from https://www.terraform.io/downloads.html
88 Initializing the backend...
89
90 Successfully configured the backend "azurerm"! Terraform will automatically
91 use this backend unless the backend configuration changes.
```

32. The code change merged to main before the intent was ascertained, but in this case there is no need to roll back.

Comparison with Lab 6b

33. Side-by-side, comparison of this and the previous lab:

Aspect	Lab 6b	Lab 7
Promotion control	Strong	Strong
Approval gates	Yes	Yes
Execution safety	Yes	Yes
Explicit intent artifact	no	Yes
Audit clarity	Weak	Strong

34. You should conclude:

- Promotion is now **declared before execution**
- Automation cannot act without intent
- The change decision exists as a **durable artifact**
- Approval now authorises **execution of a declared decision**
- Audit questions can be answered cleanly

You have shown that:

- automation without intent is dangerous
- approvals without declarations are ambiguous
- execution without explicit decision fails audits

This is the difference between:

“We ran the pipeline” and *“We decided to promote production.”*

Reflection

In real enterprises, failures rarely come from broken automation.

They come from:

- inferred decisions
- implicit intent
- missing declarations

You have now designed a system where:

- humans decide
- automation executes
- intent is explicit
- evidence is durable

That is production-grade infrastructure governance.

Lab 6b proved we could safely change an environment, now we can prove *why* we did.

Part 2. Change Authorisation

You have redesigned the change workflow so that automation can no longer infer intent. Changes now require a durable, explicit, human-authored declaration. However, a critical governance question remains:

Who is allowed to declare promotion intent for an environment?

You will now harden the promotion model further by enforcing environment ownership.

A change will now be permitted only when:

- intent is explicitly declared
- the declaration is made by an authorised owner

Correct intent from the wrong person must fail.

By the end of this phase, you should be able to demonstrate:

“A proposed change failed because the person who declared legitimate intent was not authorised to apply the change.”

And conversely:

“The same change succeeded when declared by an authorised person.”

This enforcement must be:

- automatic
- deterministic
- auditable
- non-bypassable

A user who does no authority attempts to change PROD configuration.

The system must:

- detect the violation
- refuse execution
- explain the failure clearly

No manual intervention is allowed.

Governance Model

This phase introduces a second independent contract:

Promotion Intent

Answers: *What change is being proposed?*

Environment Authority

Answers: *Who is allowed to make the change?*

Both must be satisfied before automation may execute....

Change Type	What the Human Did	What the Pipeline Concludes	Authorisation
ENV_ROOT_CHANGE	Changed environment config only (no module changes)	Local environment modification	Allowed with authority
PROMOTION_INTENT	Changed pinned module version and declared matching intent	Explicit, approved promotion	Allowed with authority
IMPLICIT_PROMOTION_ATTEMPT	Changed pinned module version without declaring promotion intent	Tried to promote shared code by stealth	Always blocked - Authority is irrelevant
NO-OP_PROMOTION_INTENT	Declared promotion intent without changing any module version	Recorded intent with no effective action	Always blocked - Authority is irrelevant

Enforce Authorisation in the Pipeline

An exemplar authorisation file, **owners.yml** has been provided at **artifacts/governance ...**

```
1 environments:
2   governance:
3     owners:
4       # no governance owners
5
6   dev:
7     owners:
8       - dev.user@company.com
9
10  test:
11    owners:
12      - test.user@company.com
13
14  prod:
15    owners:
16      - platform.admin@company.com
```

35. Create a top-level folder **governance** in your local **env-roots** and copy **artifacts/governance/owners.yml** into it. You will push it later along with other changes.

Enable authorisation feature in environment change pipelines

The change-{env} pipeline files have three feature levels. Level 1 is Terraform only, Level 2, the current level, is Terraform and Intent and Level 3 is Terraform, Intent and Ownership.

36. Update the three **change-{env}.yaml** files in **env-roots\azure-pipelines**, changing the **Features Phase** value from “2” to “3”, thus enabling OWNERSHIP logic.

37. From env-roots, run ...

```
cd c:\ado-repo\env-roots
git checkout main
git pull
git branch -D ownership-gates
git checkout -b ownership-gates
git add governance/
git add .azure-pipelines\change-dev.yaml
git add .azure-pipelines\change-test.yaml
git add .azure-pipelines\change-prod.yaml
git commit -m "Introduce ownership enforcement gates"
git push -u origin ownership-gates
```

Create and Complete/Merge the PR

This PR introduces governance enforcement without modifying Terraform code or approval gates.

You will now test if the governance enforcement works as intended.

Test 1 - Attempt an Unauthorised Promotion

37. The currently authorised owners of your envs, as stated in **owners.yaml** does not include your current user account and therefore you are not authorised to trigger a promotion to any environment at present.

38. Declare valid promotion intent for PROD, downgrading the security module version from v.1.0.2 to v1.0.1 ...

- In **envs/prod/prod-promotion.tfvars**, set the approved **security** module version to **v1.0.1**
- In **envs/prod/main.tf**, set the pinned **security** module version to **v1.0.1**

39. Push the changes ...

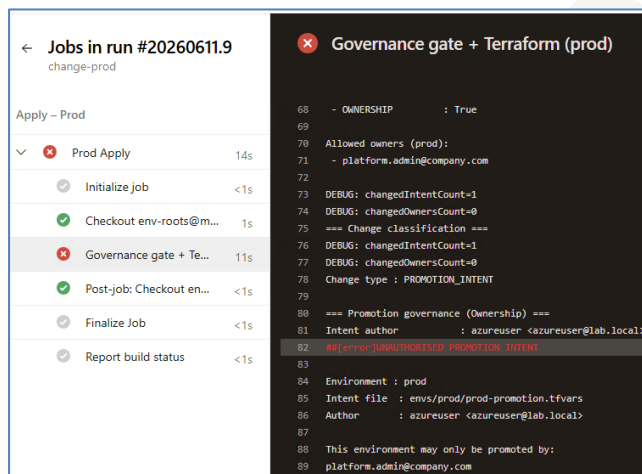
```
cd c:\ado-repo\env-roots
git checkout main
git pull
git branch -D promote-sec-v1.0.1-prod
git checkout -b promote-sec-v1.0.1-prod
git add envs\prod\prod-promotion.tfvars
git add envs\prod\main.tf
```

```
git commit -m "Promote Secv1.0.1 to PROD"
git push -u origin promote-sec-v1.0.1-prod
```

40. Raise the PR, triggering **pr-plan-prod** pipeline run
41. Complete/merge to trigger **change-prod** pipeline run
42. Grant approval and observe pipeline execution jobs
43. Confirm that the pipeline fails before Terraform execution ...
 - The failure is automatic
 - No approvals can override it

The failure message clearly explains ...

- Who attempted the promotion
- Who is authorised
- Why execution was blocked



44. Whilst the pipeline run failed, the commit to main succeeds, so roll it back;

```
git checkout main
git pull
git revert -m 1 HEAD --no-edit
git push
```

45. This will trigger the **change-prod** pipeline to run again, and it is expected to fail again but the reset will have been committed to main which will again show v1.0.2 as the security module version for PROD. You can let the pipeline run to completion/failure or cancel it. You are good to try again.

Test 2 - Perform an Authorised Promotion

Now you will add yourself as an **authorised owner** and repeat the process. At this stage, you will add yourself as an authorised owner of the PROD environment.

Update PROD ownership

45. Update **prod owners** in **envs-root/governance/owners.yml** to include your authorized ado account email address; **azureuser@lab.local**

46. Run ...

```
cd c:\ado-repo\env-roots
git checkout main
git pull
git branch -D update-ownership
git checkout -b update-ownership
git add governance\owners.yml
git commit -m "Update PROD ownership"
git push -u origin update-ownership
```

47. Raise the PR

48. **Complete/commit** the change.

Test PROD ownership

49. In **envs/prod/prod-promotion.tfvars**, (re)set the approved security module version to **v1.0.1**

50. In **env/prod/main.tf**, (re)set the pinned security module version to **v1.0.1**

```
cd c:\ado-repo\env-roots
git checkout main
git pull
git branch -D promote-sec-v1.0.1-prod-1
git checkout -b promote-sec-v1.0.1-prod-1
git add envs\prod\prod-promotion.tfvars
git add envs\prod\main.tf
git commit -m "Promote Secv1.0.1 to PROD"
git push -u origin promote-sec-v1.0.1-prod-1
```

51. Raise the PR, triggering **pr-plan-prod** pipeline run

52. Complete/merge to trigger **promote-prod** pipeline run

53. Grant approval and observe pipeline execution jobs

The screenshot displays the Azure DevOps pipeline interface. On the left, a table titled 'Jobs in run #20260611.11' shows the sequence of jobs: 'Prod Apply' (51s), 'Initialize job' (<1s), 'Checkout env-roots@m...' (2s), 'Governance gate + Te...' (48s), 'Post-job: Checkout en...' (<1s), 'Finalize Job' (<1s), and 'Report build status' (<1s). The 'Governance gate + Terraform (prod)' job is highlighted. On the right, the console output for this job is shown, including the promotion governance intent, validation steps, and Terraform execution details. A yellow arrow points to the 'Ownership validation PASSED' message in the console output.

Job Name	Duration
Prod Apply	51s
Initialize job	<1s
Checkout env-roots@m...	2s
Governance gate + Te...	48s
Post-job: Checkout en...	<1s
Finalize Job	<1s
Report build status	<1s

```
81 === Promotion governance (Ownership) ===
82 Intent author      : azureuser@lab.local>
83 Ownership validation PASSED.
84
85 === Promotion intent validation ===
86 Declared environment : prod
87 Approved module refs:
88   - network = v1.0.2
89   - security = v1.0.1
90 Pinned ref (network) : v1.0.2
91 Pinned ref (security) : v1.0.1
92 Promotion intent validated successfully.
93
94 === Terraform execution ===
95 Change type (final) : PROMOTION_INTENT
96
97 Terraform v1.18.5
```

54. In the Portal, confirm the manager of the NSG in PROD, reverts from Oliver Walsh (v1.0.2) to Peter Smith (v1.0.1)

Expected Result

- Ownership validation passes
- Intent validation passes
- Promotion proceeds normally

Run your own tests (Optional)

Test Matrix

#	Own ?	Change	Expected Result
1	N	None-module related config change	ENV ROOT CHANGE
2	N	Update module refs without corresponding update to Intent file	IMPLICIT PROMOTION INTENT
3	N	Update Intent file <i>without</i> matching module ref updates in config	NO-OP PROMOTION INTENT
4	N	Update Intent file <i>with</i> matching module ref updates in config	PROMOTION INTENT
5	Y	None-module related config change	ENV ROOT CHANGE
6	Y	Update module refs without corresponding update to Intent file	IMPLICIT PROMOTION INTENT
7	Y	Update Intent file <i>without</i> matching module ref updates in configuration	NO-OP PROMOTION INTENT
8	Y	Update Intent file <i>with</i> matching module ref updates in configuration	PROMOTION INTENT + file flag if changes are outside main.tf

55. Conduct selective tests based on the Test Matrix above.

56. Try any other configuration scenario that you feel might occur.

57. **Roll-back any Apply failures** otherwise they will be committed to main and taint it (this issue will be addressed soon)

```
git checkout main
git pull
git revert -m 1 HEAD --no-edit
```


`git push`

Then run or cancel the triggered no-op change-`{env}` pipeline run

Verification Checklist

You should be able to demonstrate all of the following:

- Module changes succeed when intent is declared correctly by an authorised owner
- Non-module change attempts succeed if attempted by an authorised user
- Terraform never runs any unauthorised change attempts
- Failure messages are explicit and audit-friendly

Shift governance out of main

Up to now governance has been enforced at apply-time. This allowed you to see governance failures clearly, observe their effects, and understand exactly what was being blocked and why. The downside of this approach is that failed governance checks occur **after** changes are merged to main, requiring rollbacks to restore the previous state. That discomfort was intentional.

In more mature delivery models, this situation is avoided by running the same governance checks **during pull request validation**, preventing unauthorised or malformed changes from ever reaching main.

From this point forward, we move governance earlier in the lifecycle — not by changing the rules, but by changing **where** they run.

Update Plan Pipelines

58. Update the three `pr-plan-{env}.yaml` files in `env-roots\azure-pipelines`, changing the **Features Phase** value from “1” to “3”, thus enabling OWNERSHIP logic.

59. Run ...

```
cd c:\ado-repo\env-roots
git checkout main
git pull
git branch -D plan-updates
git checkout -b plan-updates
git add .azure-pipelines\pr-plan-*.yaml
git commit -m "Update plan pipelines"
git push -u origin plan-updates
```

60. Raise the PR and complete/merge the changes to main

This PR introduces early governance enforcement without modifying Terraform code or approval gates.

These plan pipelines are triggered by pull requests. Should the governance gate block progression then we must prevent merging to main. This is achieved by branch policies on

main. We already have these in-place, but they are currently “Optional”, we need to make them “Required”

61. In DevOps, open **Repos > Branches > main > More options > Branch policies**

62. Edit each of the existing Build validations, changing ‘Policy requirement’ from “Optional” to **“Required”**

Build Validation 3			
Validate code by pre-merging and building pull request changes.			
Enabled	Name	Path filter	Trigger
☒ On	pr-plan-dev Required	/envs/dev/**	Automatic Expires after 12 hours
☒ On	pr-plan-prod Required	/envs/prod/**	Automatic Expires after 12 hours
☒ On	pr-plan-test Required	/envs/test/**	Automatic Expires after 12 hours

Test governance before hitting main

Attempt an env-root update in TEST - Fast Fail

63. In **envs-root/envs/test/terraform.tfvars** ...

Update the data subnet_cidrs from "10.1.2.0/24" to **“10.1.99.0/24”**

64. Run...

```
cd c:\ado-repo\env-roots
git checkout main
git pull
git branch -D test-env-change
git checkout -b test-env-change
git add .
git commit -m "Update environment"
git push -u origin test-env-change
```

65. Create new PR,

66. Grant permissions and then observe the governance-integrated plan pipeline run, and fail ...

attempt update to test

Active 1155 Michael Coulling-Green proposes to merge test-env-change into main

Overview Files Updates Commits

1 required check failed

pr-plan-test Build failed

Governance gate + terraform plan (test) / Governance gate (PR validation)

51 UNAUTHORISED ENVIRONMENT CHANGE: actor is not an owner of test.

60 Script failed with exit code: 1

No merge conflicts

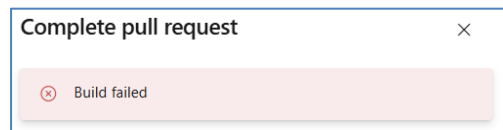
Last checked Just now

Description

attempt update to test

67. This failure is BEFORE the merge commit and branch policy on main and prevents any commit attempts.

68. Attempt to complete the merge ...



69. Whilst the merge fails, the new branch still remains in the repo. **Abandon** the PR in ADO and the clean up by running ...

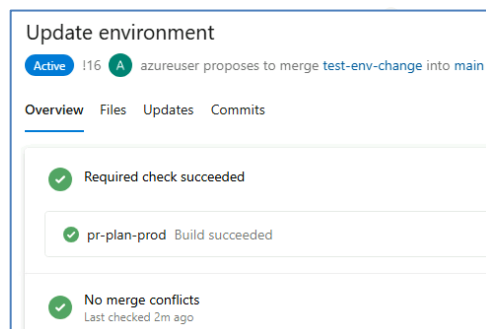
```
git switch main
git push origin --delete test-env-change
git branch -D test-env-change
```

Test success before hitting main

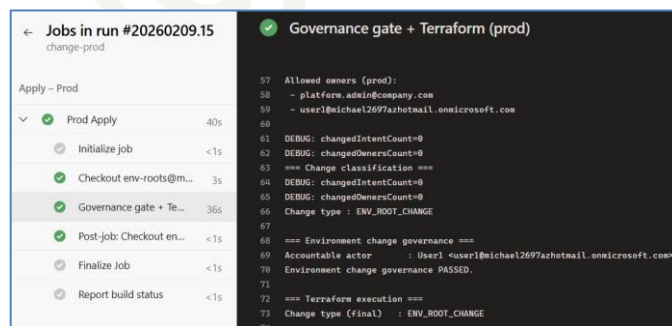
Attempt an env-root update in PROD

70. Using the steps from the previous test, adjusted to reference the **PROD** environment, attempt to update the **PROD data** subnet_cidrs from "10.2.99.0/24" to "**10.2.2.0/24**"

71. The **pr-plan-prod** pipeline should pass governance checks...



72. Confirm/merge to trigger the **change-prod** pipeline, approving the run. This pipeline also runs governance checks before proceeding ...



Key Takeaway

By the end of this lab, you should understand that:

- Local environment changes can safely use the commit itself as intent
- The explicitness of intent must scale with blast radius
- Cross-environment promotions require explicit, scoped approval.
- Environment ownership governs **all** changes to that environment — not just module upgrades.

At this point:

- Module promotion is governed
- Environment configuration changes are governed

This raises one final question: *Who governs changes to the governance itself?*

That question is answered next.

Part 3 - Governance-of-Governance

This part of the lab introduces and enforces a governance-of-governance pipeline, allowing you to observe how **existing governance state** affects enforcement, including bootstrap failure and break-glass recovery.

Phase 1 – Introduce the Governance Pipeline

38. You have been provided with a ready-made governance pipeline file at **artifacts\governance\governance.yml**. Copy this file into **envs-root\azure-pipelines** and push it to ADO ...

```
cd c:\ado-repo\env-roots
git checkout main
git pull
git branch -D gov-pipeline
git checkout -b gov-pipeline
git add .azure-pipelines\governance.yml
git commit -m "Governance pipeline"
git push -u origin gov-pipeline
```

39. Add the PR and Complete/merge to main

40. In DevOps > Pipelines > Create Pipeline ...

- Azure Repos Git
- Repository: **env-roots**
- Existing YAML file
- Branch: **main**
- Path: **.azure-pipelines/governance.yml**

41. Save the pipeline without running it.

42. Rename the pipeline from 'env-roots' to '**gov-of-gov**'

At this point, the pipeline exists, but nothing is enforcing it.

Do not run it manually

43. Open `governance/owners.yml` on main.

Confirm:

```
environments:  
  governance:  
    owners:  
      # no governance owners
```

At this moment:

- Governance is defined
- Governance protection exists
- Governance is **not enforced**

Phase 2 – Enforce Governance-of-Governance

To ensure a PR is required before committing changes of `owners.yml` to main, we attach a required build validation policy

44. DevOps > Repos > Branches

45. Locate branch: **main**

46. Select ... More options → **Branch policies**

47. Under Build validation, click **Add**

48. Configure:

- **Build pipeline:** `gov-of-gov`
- **Path filter (optional):** `/governance/owners.yml`
- **Trigger:** Automatic
- **Policy requirement:** Required

49. Click **Save**

This is the exact moment the system becomes self-protecting.

Attempt Normal Changes (Expected Failure)

50. Modify `governance\owners.yml`, attempt to add a new **dev** owner email.

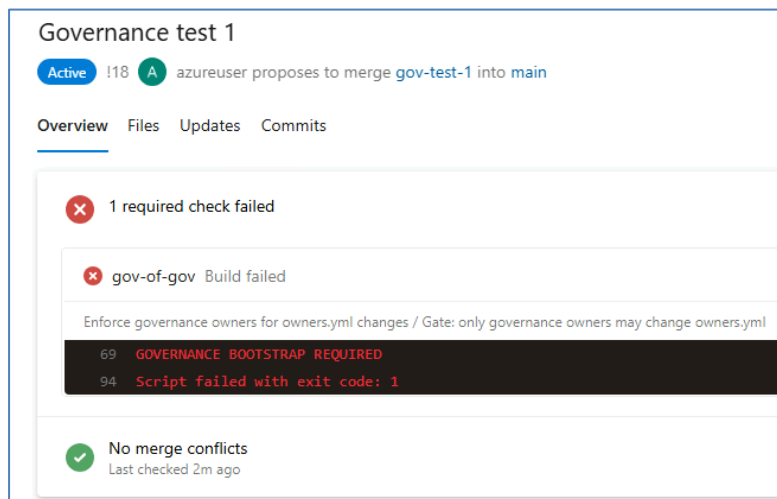
```
cd c:\ado-repo\env-roots  
git checkout main  
git pull  
git branch -D gov-test-1  
git checkout -b gov-test-1  
git add .  
git commit -m "Governance test 1"
```

```
git push -u origin gov-test-1
```

51. Open the PR to main

Expected result:

- gov-of-gov pipeline runs
- Gate fails with:



52. Abandon the PR and roll-back ...

```
git switch main
git push origin --delete gov-test-1
git branch -D gov-test-1
```

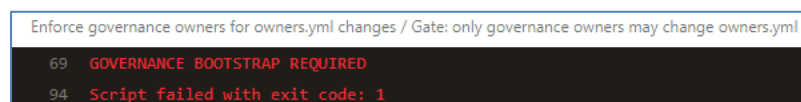
53. Add yourself (**azureuser@lab.local**) as a governance owner in **governance\owners.yml** and attempt to commit ...

```
cd c:\ado-repo\env-roots
git checkout main
git pull
git branch -D gov-test-2
git checkout -b gov-test-2
git add .
git commit -m "Governance test 2"
git push -u origin gov-test-2
```

54. Open the PR to main.

Expected result; Same failure, same bootstrap message

You cannot grant yourself authority through a governed process...



55. Abandon the PR and roll-back ...

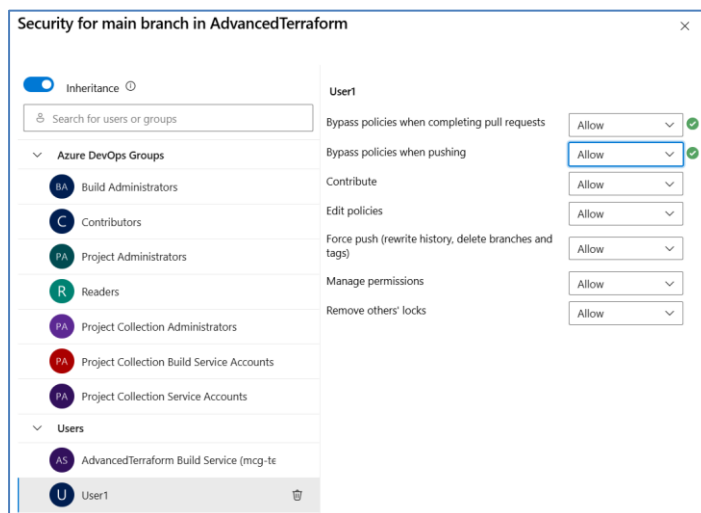
```
git switch main
git push origin --delete gov-test-2
git branch -D gov-test-2
```

Phase 3 – Break-Glass Recovery

56. In ADO, locate branch: **main**

57. Select ... More options → Branch security ...

- Allow your account **Bypass policies** permissions ...



58. Edit **owners.yml** *directly* in main, add your email address as a governance owner and commit directly.

Every real governance system needs a break-glass. The danger is pretending it doesn't. Roll back the branch permissions now that the fix is in place.

Phase 4 – Normal Governance Operation

59. **Sync main** for the updated owners.yml ...

```
git checkout main
git pull
```

60. Retry a **dev** ownership change following the earlier steps.

Expected result:

- Governance gate passes
- PR can be merged

✓ Governance-of-gover...
1 job completed 18s

✓ Enforce governance owners ... 1.

Rerun Stage

```
Governance owners (from origin/main):  
- user1@michael2697azhotmail.onmicrosoft.com  
  
[GOV-OWNERSHIP] Actor is a governance owner - OK.  
Governance-of-governance gate PASSED.
```

61. Complete/merge the PR

Key Learning Points

- Governance pipelines must exist **before** enforcement
- Enabling enforcement can expose **historical governance debt**
- Empty governance is a valid state - but **not a safe one**
- Break-glass access is mandatory, not optional

*** Congratulations, you have completed this lab ***

Copyright

© 2026 QA Michael Coulling-Green. All rights reserved. These lab materials are provided as part of an authorised training course. Course attendees may reuse these materials for their own personal learning and reference outside of the training session. Unauthorised copying, redistribution, publication, or use of these materials to deliver training is prohibited without prior written permission from the author.

© QA 2026